

Tecnológico de Monterrey CEM
Cierre de ETAPA 2 del RETO.

Carrera: Ingeniería en Tecnologías Computacionales (ITC)

Equipo: 12

Implementación de internet de las cosas (Gpo 501)
Profesores | Benito Granados Rojas, David Higuera Rosales, Roberto Martínez Román, Ariel Ortíz Ramírez

3 de Noviembre de 2026

Sergio Antonio Foyo Puga | A01805448
José Andres Gutierrez Muñoz | A01804425

MONTAJE Y PRIMERA LECTURA CÓDIGO DEL SKETCH DE PRUEBA USADO

DS18B20

El DS18B20 es un sensor de temperatura el cual cuenta con dos presentaciones, una siendo un sensor de temperatura normal y la otra cuenta con un cable el cual se puede usar para medir las temperaturas de los líquidos.

El código usado nos ayuda a medir la temperatura utilizando la transferencia de información de tecnología one wire del sensor. Después, da como salida el resultado obtenido en celsius.

```
#include <OneWire.h>

#include <DallasTemperature.h> // Librerias del sensor

#define ONE_WIRE_BUS 4 // Pin de datos

OneWire oneWire(ONE_WIRE_BUS);

DallasTemperature sensors(&oneWire);

void setup() {

  Serial.begin(115200); // Inicia el monitor serial

  sensors.begin(); // Inicia el sensor



  Serial.println("DS18B20 Prueba de temperatura");

}



void loop() {

  sensors.requestTemperatures(); // Solicita la temperatura

  float temperature = sensors.getTempCByIndex(0); // Obtiene la lectura
```

```
  if (temperature == DEVICE_DISCONNECTED_C) {
    Serial.println("No se detecto el sensor"); // Si no detecta el
sensor se deja un mensaje

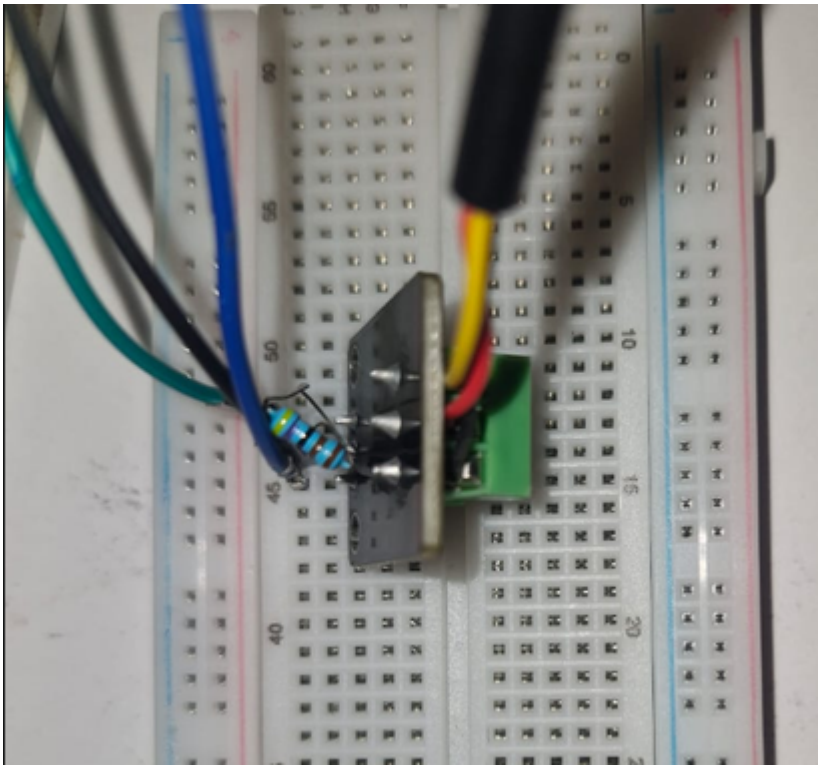
  } else {
    Serial.print("Temperatura: ");
    Serial.print(temperature); // Muestra la temperatura
    Serial.println(" °C");
  }

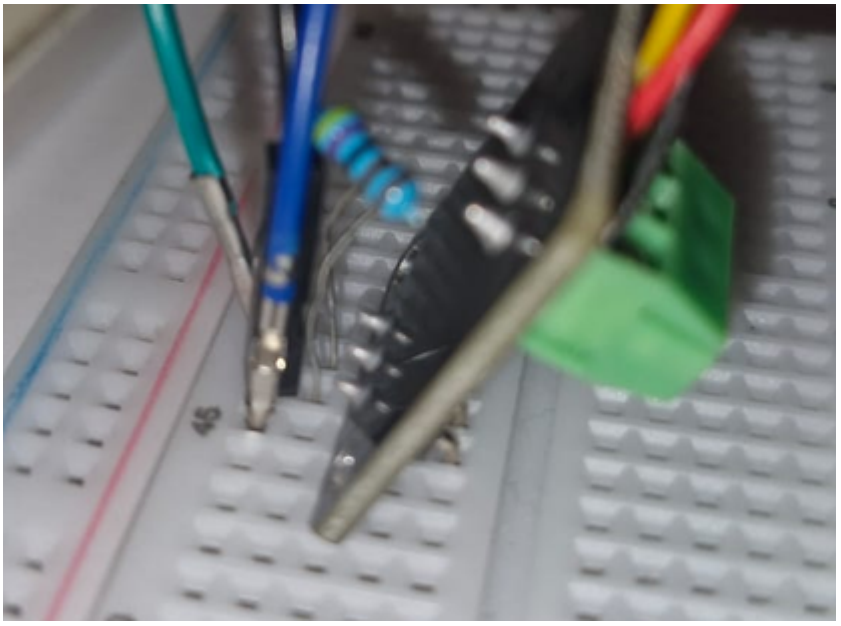
  delay(1000); // Espera un segundo
}
```

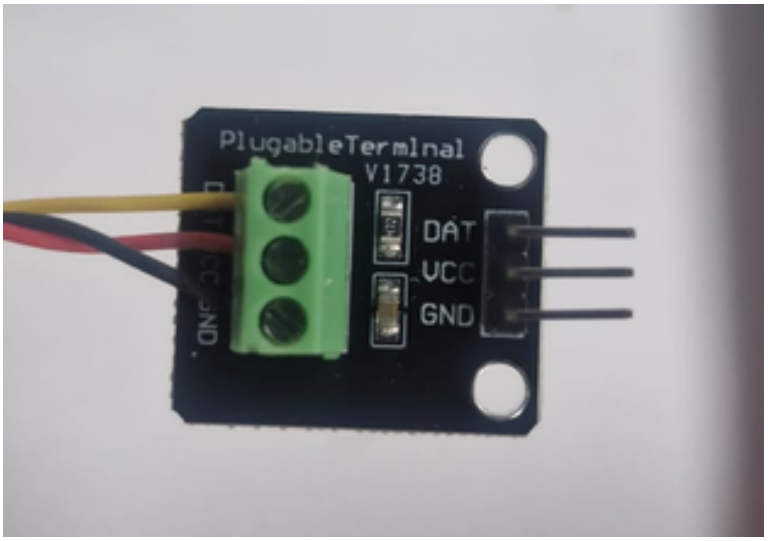
Cambio gradual de temperatura al sostener el sensor con la mano:

```
08:27:49.505 -> Temperatura: 18.31 °C
08:27:50.532 -> Temperatura: 18.31 °C
08:27:51.560 -> Temperatura: 18.31 °C
08:27:52.590 -> Temperatura: 18.31 °C
08:27:53.617 -> Temperatura: 18.50 °C
08:27:54.645 -> Temperatura: 19.12 °C
08:27:55.674 -> Temperatura: 19.75 °C
08:27:56.701 -> Temperatura: 20.37 °C
08:27:57.728 -> Temperatura: 20.94 °C
```

Conexiones hechas para el funcionamiento del sensor







Listado de conexiones Sensor -> ESP32.

GND -> GND

VCC -> 3V3

DAT -> GPIO4

Nota: Se conectó el puerto DAT y VCC con una resistencia de 4,700 ohmios, la que permite una mejor comunicación con el sensor, sirviendo como un resistor pull-up.

MAX30102

El sensor usa dos longitudes de onda, porque la hemoglobina oxigenada y la desoxigenada absorben la luz de maneras distintas. La hemoglobina desoxigenada absorbe mucho más luz roja, mientras que la otra absorbe más luz infrarroja, el sensor compara la cantidad de luz reflejada que recibe su fotodetector en ambas longitudes de onda y con un algoritmo de cálculo y calibración logra estimar el porcentaje de hemoglobina que transporta oxígeno en la sangre.

El código usado realiza cálculos a partir de las mediciones obtenidas por el sensor para dar como salida los latidos por minuto (BPM) y la saturación de oxígeno en la sangre.

```
#include <Wire.h>
```

```
#include "MAX30105.h"
#include "spo2_algorithm.h" // Algoritmo de BPM y SpO2
MAX30105 sensor;
#define TAM_BUFFER 100

uint32_t bufferInfrarrojo[TAM_BUFFER];
uint32_t bufferRojo[TAM_BUFFER];
int32_t spo2;
int32_t frecuenciaCardiaca;
int8_t spo2Valido;
int8_t frecuenciaCardiacaValida;



void setup() {

  Serial.begin(115200);

  Wire.begin(21, 22);

  if (!sensor.begin(Wire, I2C_SPEED_STANDARD)) {

    Serial.println("¡Sensor no detectado!");

    while (1);

  }

  sensor.setup(60, 4, 2, 100, 411, 4096);
```

```
  Serial.println("Prueba MAX30102");

  for (int i = 0; i < TAM_BUFFER; i++) {

    while (!sensor.available()) sensor.check();

    bufferRojo[i] = sensor.getRed();

    bufferInfrarrojo[i] = sensor.getIR();

    sensor.nextSample();

  }

}

void loop() {

  for (int i = 25; i < TAM_BUFFER; i++) {

    bufferRojo[i - 25] = bufferRojo[i];

    bufferInfrarrojo[i - 25] = bufferInfrarrojo[i];

  }

  for (int i = 75; i < TAM_BUFFER; i++) {
```

```
    while (!sensor.available()) sensor.check();

    bufferRojo[i] = sensor.getRed();

    bufferInfrarrojo[i] = sensor.getIR();

    sensor.nextSample();

  }

  maxim_heart_rate_and_oxygen_saturation(

    bufferInfrarrojo, TAM_BUFFER, bufferRojo,

    &spo2, &spo2Valido, &frecuenciaCardiaca, &frecuenciaCardiacaValida

  );

  if (frecuenciaCardiacaValida && spo2Valido) {

    Serial.print("BPM: ");

    Serial.print(frecuenciaCardiaca);

    Serial.print(" | SpO2: ");

    Serial.print(spo2);

    Serial.println("%");
```

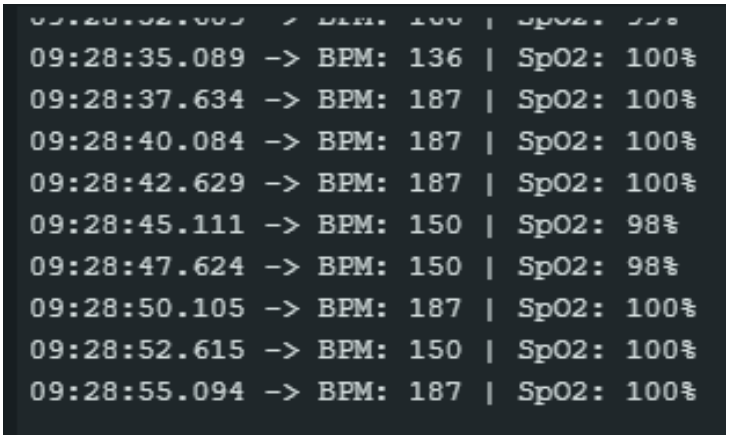
```
  } else {

    Serial.println("Leyendo...");

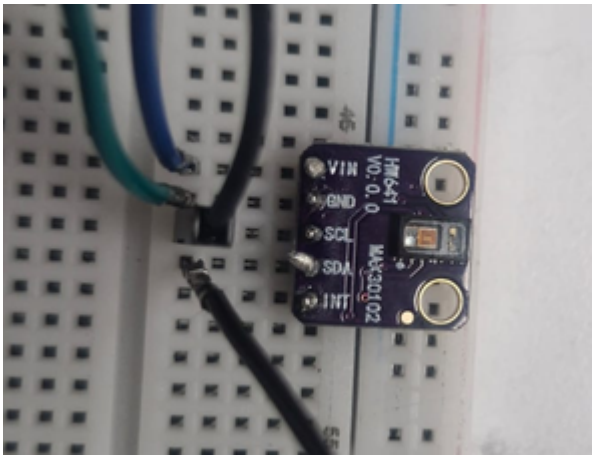
  }

}
```

Cambio de BPM (latidos por minuto por sus siglas en inglés):



Nota: *El nivel del ruido del MAX30102 es alto y muy notorio, esto se debe a la presión que tiene el dedo contra el sensor, una presión que varía dependiendo de la fuerza que se aplique. Como es imposible ajustar a un valor constante de presión, este muestra valores inadecuados como 187 LPM. Para esto, se debería crear una solución para mantener este lector a una presión adecuada y constante. Sus lecturas se mantienen constantes, lo que demuestra su funcionamiento, esto es, sin considerar los valores altos de LPM.*



Listado de conexiones Sensor -> ESP32.

GND -> GND

VIN -> 3V3

SDA -> GPIO21

SCL -> GPIO22

MLX90614

El MLX90614 mide la temperatura mediante infrarrojos sin contacto directo con la piel. Eso facilita su integración en una pulsera y puede brindar mayor comodidad. En cambio el DS18B20 no necesita tocar la piel para realizar mediciones, aunque sus lecturas pueden verse alteradas por el movimiento y la temperatura ambiental.

Este código captura las mediciones del sensor, la medición de objeto que cubre la luz infrarroja y la temperatura ambiente, la cual es la temperatura del chip y el aire que lo rodea. Luego, da como salida la temperatura de los dos.

```
#include <Wire.h>

#include <Adafruit_MLX90614.h> // Libreria del sensor



Adafruit_MLX90614 sensor = Adafruit_MLX90614();
```

```ino
void setup() {
  Serial.begin(115200); // Inicia serial
  Wire.begin(21, 22); // Pines I2C

  if (!sensor.begin()) {
    Serial.println("No se detecto el sensor.");
    while (1);
  }

  Serial.println("MLX90614 Prueba de temperatura");
}

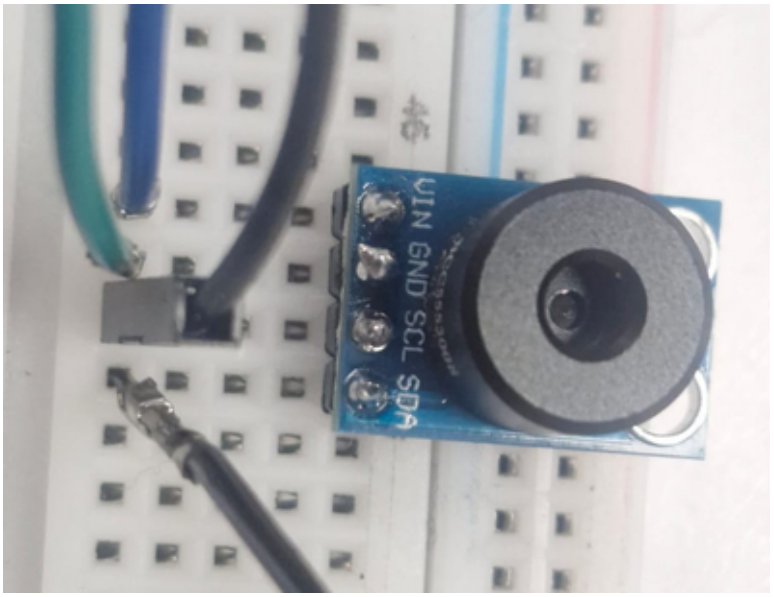
void loop() {
  float ambiente = sensor.readAmbientTempC(); // Temperatura ambiente
  float objeto = sensor.readObjectTempC(); // Temperatura del objeto

  Serial.print("Ambiente: ");
  Serial.print(ambiente);
  Serial.print(" °C | Objeto: ");
  Serial.print(objeto);
  Serial.println(" °C");

  delay(1000); // Espera un segundo
}
```

Cambio de temperatura relativo, midiendo la temperatura exterior y la del objeto (dedo) que cubre el sensor.

```
09:47:29.571 -> Ambiente: 25.21 °C | Objeto: 28.97 °C
09:47:30.571 -> Ambiente: 25.31 °C | Objeto: 28.85 °C
09:47:31.570 -> Ambiente: 25.39 °C | Objeto: 28.77 °C
09:47:32.568 -> Ambiente: 25.49 °C | Objeto: 28.69 °C
09:47:33.561 -> Ambiente: 25.57 °C | Objeto: 28.75 °C
09:47:34.560 -> Ambiente: 25.63 °C | Objeto: 28.69 °C
09:47:35.562 -> Ambiente: 25.71 °C | Objeto: 28.63 °C
09:47:36.560 -> Ambiente: 25.77 °C | Objeto: 28.69 °C
09:47:37.588 -> Ambiente: 25.83 °C | Objeto: 28.59 °C
```



Listado de conexiones Sensor -> ESP32.

GND -> GND

VIN -> 3V3

SDA -> GPIO21

SCL -> GPIO22

MPU6050

El MPU6050 cuenta con un acelerómetro de 3 ejes y con un giroscopio de 3 ejes, conformando los 6 ejes que tiene este sensor. También cuenta con un Procesador de

Movimiento Digital (o DMP por sus siglas en inglés), el cual puede realizar algunos cálculos complejos, lo que libera un poco la carga de trabajo del microcontrolador.

Este código mide la aceleración en los ejes X, Y, Z con su acelerómetro, junto con la aceleración de rotación en los ejes con su giroscopio. Luego, este da como salida las dos mediciones.

```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h> // Librerias del sensor

Adafruit_MPU6050 sensor;

void setup() {
  Serial.begin(115200); // Inicia serial
  Wire.begin(21, 22); // Pines I2C

  if (!sensor.begin()) {
    Serial.println("No se detecto el sensor");
    while (1);
  }

  Serial.println("MPU6050 Prueba");
}

void loop() {
  sensors_event_t accel, gyro, temp;
  sensor.getEvent(&accel, &gyro, &temp); // Obtiene lecturas

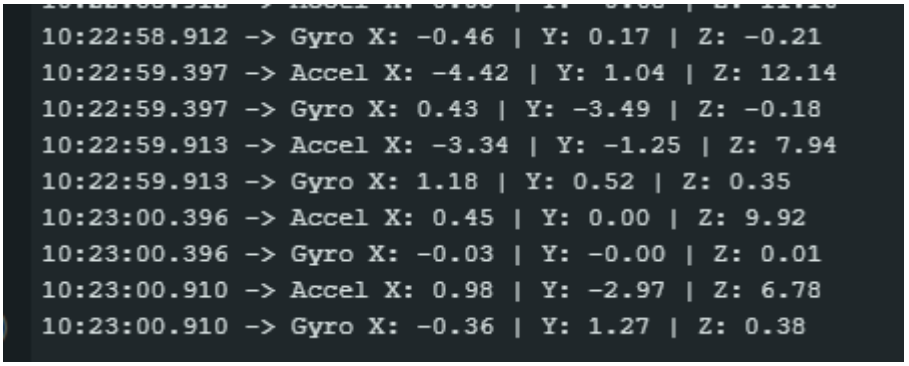
  Serial.print("Aceleracion X: ");
  Serial.print(accel.acceleration.x);
```

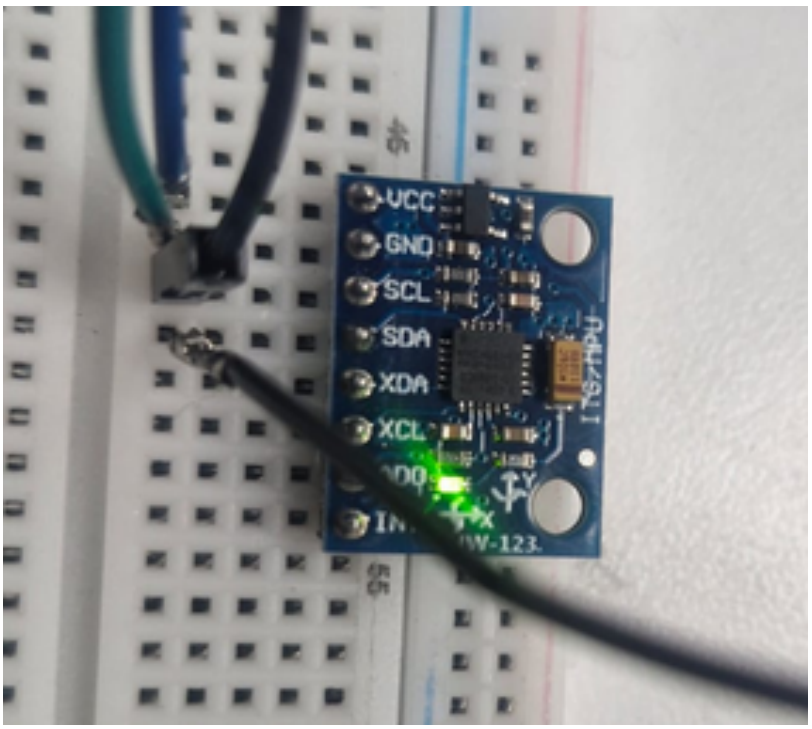
```
  Serial.print(" | Y: ");
  Serial.print(accel.acceleration.y);
  Serial.print(" | Z: ");
  Serial.println(accel.acceleration.z);


  Serial.print("Gyro X: ");
  Serial.print(gyro.gyro.x);
  Serial.print(" | Y: ");
  Serial.print(gyro.gyro.y);
  Serial.print(" | Z: ");
  Serial.println(gyro.gyro.z);


  delay(500); // Espera medio segundo
}
```

Lectura del acelerómetro en los 3 ejes tridimensionales X, Y, Z, aplicando un movimiento leve de rotación.





Listado de conexiones Sensor -> ESP32.

GND -> GND

VCC -> 3V3

SDA -> GPIO21

SCL -> GPIO22

MQ135

El MQ135 mide la calidad del aire mediante su interior, ya que tiene un pequeño tubo cerámico recubierto con dióxido de estaño (SnO_2), el cual funciona como material sensible. Al estar en presencia de ciertos gases, este cambia su conductividad eléctrica ya que reacciona el oxígeno absorbido por el sensor con moléculas del gas, lo que provoca que este baje su resistencia y suba su conductividad eléctrica.

El código usado mide la calidad del aire mediante la medición usando el pin analógico del sensor. Este código únicamente da la medición del sensor, no clasifica ni ajusta adecuadamente la medición obtenida.Mientras más alto sea el valor, peor es la calidad del aire a su alrededor.

```
#define MQ135 34 // Pin analogico


void setup() {
  Serial.begin(115200); // Inicia serial
  analogReadResolution(12); // Resolucion de 12 bits


  Serial.println("Prueba MQ-135");
}

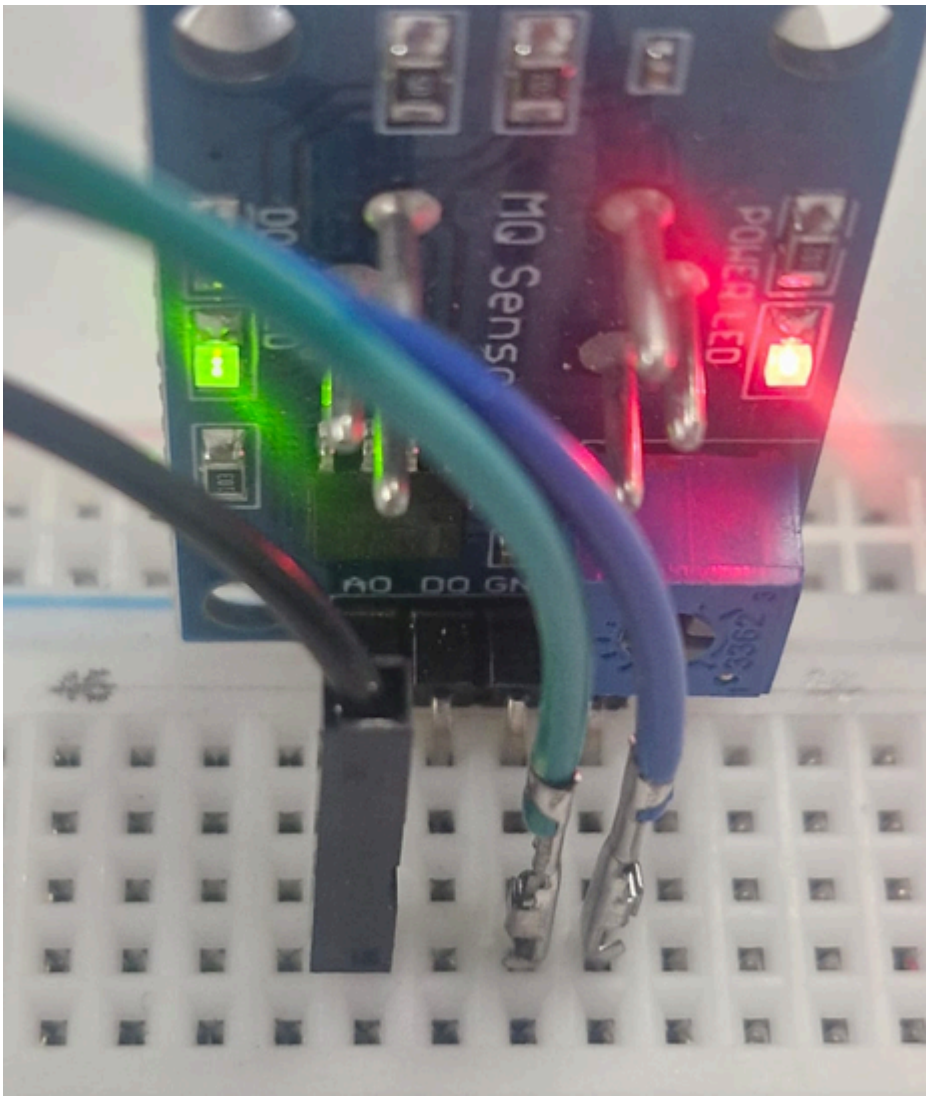

void loop() {
  int lectura = analogRead(MQ135); // Lee el sensor
  float voltaje = lectura * (3.3 / 4095.0); // Calcula voltaje


  Serial.print("Lectura: ");
  Serial.print(lectura);
  Serial.print(" | Voltaje: ");
  Serial.print(voltaje);
  Serial.println(" V");


  delay(1000); // Espera un segundo
}
```

Prueba de lectura de valores básica, antes de ser modificada adecuadamente para calcular calidad del aire.

```
10:35:05.523 -> Lectura: 894 | Voltaje: 0.72 V
10:35:06.524 -> Lectura: 883 | Voltaje: 0.71 V
10:35:07.520 -> Lectura: 880 | Voltaje: 0.71 V
10:35:08.523 -> Lectura: 896 | Voltaje: 0.72 V
10:35:09.524 -> Lectura: 874 | Voltaje: 0.70 V
10:35:10.524 -> Lectura: 862 | Voltaje: 0.69 V
10:35:11.524 -> Lectura: 864 | Voltaje: 0.70 V
10:35:12.524 -> Lectura: 842 | Voltaje: 0.68 V
10:35:13.553 -> Lectura: 839 | Voltaje: 0.68 V
```



Listado de conexiones Sensor -> ESP32.

GND -> GND

VCC -> VIN

A0 -> GPIO34

Nota: Para este caso no se usó un divisor de voltaje ya que las mediciones se encontraban muy bajas por el ambiente en el que se realizó la prueba. Es recomendable usar un divisor de voltaje para evitar daños a la ESP32, ya que algunos modelos, como el que se usó en este proyecto, no tienen la capacidad de tolerar voltajes mayores a 3.3 voltios.

Para mejorar la accesibilidad del aparato, independientemente de los sensores que se usen, un actuador útil sería un pequeño altavoz, que permita emitir alertas sonoras con diferentes tonos para avisar sobre eventos importantes a personas con discapacidad visual. También puede utilizarse para reproducir mensajes de voz que indiquen el tipo de alerta por la cual avisa. Esto mejoraría la accesibilidad a las personas, lo que crearía un producto más fácil de usar, ya sea una persona sin discapacidad o una persona con algún tipo de discapacidad que le impide escuchar o ver el monitor.

JUSTIFICACIONES Y EVIDENCIAS DE ESP-32 CONECTADO A WIFI

Se decidió elegir como forma de conexión WiFi 802.11 b/g/n/ac, estándar creado por el Instituto de Ingenieros de Electricidad y Electrónica, o IEEE por sus siglas en inglés. Opera principalmente en la banda de 2.4GHz. La versión WiFi n (802.11n) también puede operar en la banda de 5GHz. Otras bandas más nuevas, como lo es WiFi 6E, opera en la frecuencia de 6GHz.

Otras alternativas usadas en el mercado, sus motivos para ser consideradas y sus motivos por ser rechazadas, son las siguientes:

- Zigbee: Opera a 2.4GHz a nivel mundial, aunque también puede funcionar en 915 MHz en América y 868 MHz en Europa. Zigbee permite varias topologías de red como de estrella, de árbol y mesh. Cuenta con encriptación de AES - 128 bits como método de seguridad, lo que la hace ideal para dispositivos de IoT.

- Bluetooth: Opera en la banda de 2.4 GHz, utilizando la banda ISM (Industrial, Scientific and Medical). Tanto Bluetooth clásico como Bluetooth Low Energy (BLE) funcionan en esta frecuencia. BLE, introducido por primera vez en 2010 con la especificación de Bluetooth 4.0, manda paquetes de información de manera menos frecuente, lo que permite ahorrar batería de manera eficiente. Otra diferencia es que, BLE permite la conexión de más de 7 dispositivos de tipo esclavo, únicamente dependiendo del hardware y otros factores, mientras que Bluetooth solo permite 7 dispositivos.
- LoraWAN: Utiliza bandas de frecuencia menores a 1GHz, libres de licencia (regionales). Las más comunes son 868 MHz en Europa y 915 MHz en América. No cuenta con una licencia, lo que permite su uso libre. Al igual que Zigbee, LoraWAN cuenta con encriptación de AES - 128 bits.

Se seleccionó Wi-Fi en la banda de 2.4 GHz para el ESP32 porque permite una integración nativa y directa con la red local e internet mediante el protocolo TCP/IP, logrando eliminar la necesidad de gateways intermedias y simplificando muchísimo el diseño del hardware. Operar en esta frecuencia nos da un equilibrio óptimo entre tasa de transferencia de datos y cobertura, ofreciendo una menor atenuación al atravesar muros comparándolo con bandas más altas. Esto nos garantiza una conexión de baja latencia para la aplicación, otorgando a los usuarios una libertad total de movimiento dentro del área de cobertura sin depender de conexiones cableadas y mucho menos de un hardware adicional en sus dispositivos de control.

CÓDIGO USADO PARA LA CONEXIÓN A WIFI

```
#include <WiFi.h> // Libreria para WiFi en la ESP32 DevKitC1



// Reemplaza con los datos de tu red Wi-Fi, en este caso se utiliza
texto unicamente como ejemplo

const char* ssid = "SSID";

const char* password = "contrasena";



void setup() {

  Serial.begin(115200);
```

```
  delay(1000);


  // Iniciar en modo Station
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);


  Serial.print("Conectando a Wi-Fi");


  // Esperar a que se establezca la conexión
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

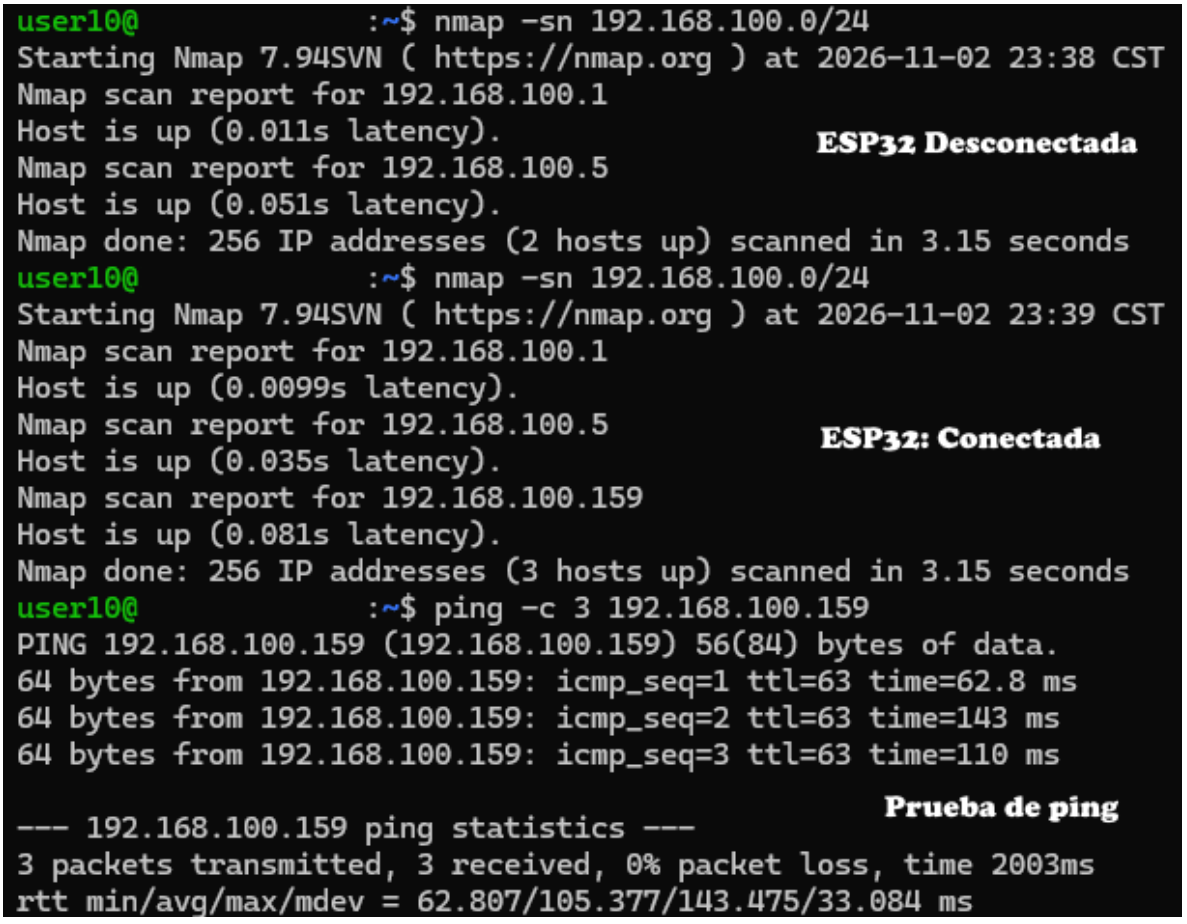

  Serial.println("\n¡Conectado exitosamente!");
  Serial.print("Dirección IP asignada: ");
  Serial.println(WiFi.localIP());
}


void loop() {
}
```

Salida del monitor serial, mostrando la IP asignada a la ESP32.

```
23:38:55.957 -> ¡Conectado exitosamente!
23:38:55.957 -> Dirección IP asignada: 192.168.100.159
23:39:20.262 -> .........
23:39:24.375 -> ¡Conectado exitosamente!
23:39:24.375 -> Dirección IP asignada: 192.168.100.159
```

Prueba de conexión con la ESP32 DevKitC1.



Se analiza la subred privada mediante la herramienta de enumeración de usuarios en la red de NMAP, demostrando una nueva conexión que se aparece en la subred después de conectar y desconectar la ESP32 de la fuente de alimentación. Después, se utiliza el comando ping, utilizando paquetes Internet Control Message Protocol (ICMP) para diagnosticar el estado de conexión de la ESP32. El comando corrió con éxito, demostrando el estado de conectividad de la ESP32 a la red WiFi. Para este caso, no se hizo ningún tipo de conexión más que la conexión de la ESP32 a la computadora.

BIBLIOGRAFÍA

- Analog Devices. (s. f.). MAX30102: High-sensitivity pulse oximeter and heart-rate sensor for wearable health. https://www.analog.com/media/en/technical-documentation/data-sheets/max30102.pdf

- Analog Devices. (s. f.). DS18B20: Programmable resolution 1-wire digital thermometer. https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf

- Ulanski, M. (8 de mayo de 2024). Bluetooth Low Energy vs. Bluetooth Classic: What's the Difference?. Ezurio. Recuperado de https://www.ezurio.com/resources/blog/bluetooth-low-energy-vs-bluetooth-classic-what-s-the-difference

- Logicbus. (19 de febrero de 2025). Zigbee Technology. Logicbus. Recuperado de https://www.logicbus.com.mx/blog/zigbee-technology/